Lesson 4

Solvers & Sklearn

Haoxiang Sun @ RUC CS



Gurobi Optimizer

Gurobi是由美国 Gurobi Optimization 公司开发新一代大规模优化器。无论在供应链、制造、金融、保险、交通、服务等各种领域，当实际问题越来越复杂，问题规模越来越庞大的时候，管理人员需要一个经过证明可以信赖的大规模优化工具，为科学决策提供质量保证，为 企业数字化转型升级提供算力支撑。在理论和实践中，Gurobi 优化工具都被证明是全球性能领先的大规模优化器，具有突出的性价比，可以为客户在开发和实施中极大降低成本。

Academic 授权可免费获取

1. 获取授权
2. 安装 gurobipy，通过python接口使用

Gurobi Optimizer

Gurobi 是全局精确优化器，支持的模型类型包括：

（1）连续和混合整数线性问题

（2）凸目标或约束连续和混合整数二次问题

（3）非凸目标或约束连续和混合整数二次问题

（4）含有对数、指数、三角函数、高阶多项式目标或约束，以及任何形式的分段约束的非线性问题

（5）含有绝对值、最大值、最小值、逻辑与或非目标或约束的非线性问题

Gurobi 技术优势：

（1）可以求解大规模线性问题，二次型问题和混合整数线性和二次型问题

（2）支持非凸目标和非凸约束的二次优化

（3）支持多目标优化，既可以支持分层多目标优化，让不同目标具备不同的优先级，也可以支持加权组合多目标优化，让不同目标通过权重组合成单一目标

（4）支持包括SUM, MAX, MIN, AND, OR等广义约束和逻辑约束

（5）支持包括高阶多项式、指数、三角函数等的广义函数约束

（6）问题尺度只受限制于计算机内存容量，不对变量数量和约束数量有限制

（7）采用最新优化技术，充分利用多核处理器优势。支持并行计算

（8）提供了方便轻巧的接口，支持 C++, Java, Python, .Net, Matlab 和R，内存消耗少

$$\max \quad -3x_1 + x_3$$

$$\text{s.t.} \begin{cases} x_1 + x_2 + x_3 \leq 4 \\ -2x_1 + x_2 - x_3 \geq 1 \\ 3x_2 + x_3 = 9 \\ x_i \geq 0 \end{cases}$$

$$\max \quad z = 2x_1 + x_2$$

$$\text{s.t.} \begin{cases} x_1 + x_2 \leq 5 \\ -x_1 + x_2 \leq 0 \\ 6x_1 + 2x_2 \leq 21 \\ x_1, x_2 \in \mathbb{Z}_+ . \end{cases}$$

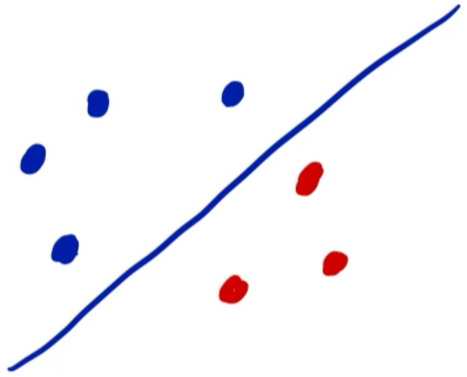
CVXPY

- 凸优化问题求解器
- 可以使用 Gurobi 作为求解器

Support Vector Machine 支撑向量机

$$\max_{\alpha} \sum_{i=1}^{m} \alpha_i - \frac{1}{2} \sum_{i,j=1}^{m} \alpha_i \alpha_j y_i y_j k(x_i, x_j)$$

$$\text{s.t.} \begin{cases} 0 \le \alpha_i \le C \\ \sum_{i=1}^{m} \alpha_i y_i = 0. \end{cases}$$



what about: $\max_{\alpha} \sum_{i=1}^{m} \alpha_i - \frac{1}{2} \sum_{i,j=1}^{m} \alpha_i \alpha_j y_i y_j k(x_i, x_j) - \theta \sum_{i=1}^{m} \alpha_i^{\gamma}$?

```
def _gram_matrix(self, X):
    n_samples, n_features = X.shape
    K = np.zeros((n_samples, n_samples))
    for i, x_i in enumerate(X):
        for j, x_j in enumerate(X):
            K[i, j] = self._kernel(x_i, x_j)
    return K
```

求解 Lagrange 乘子

```
def _compute_multipliers(self, X, y):
    n_samples, n_features = X.shape

    K = self._gram_matrix(X)
    P = np.outer(y, y) * K
    G = np.diag(np.ones(n_samples) * -1)
    h = np.ravel(np.zeros(n_samples))
    A = np.asmatrix(y).reshape(1, -1)
    b = 0.0

    x = cp.Variable(n_samples)

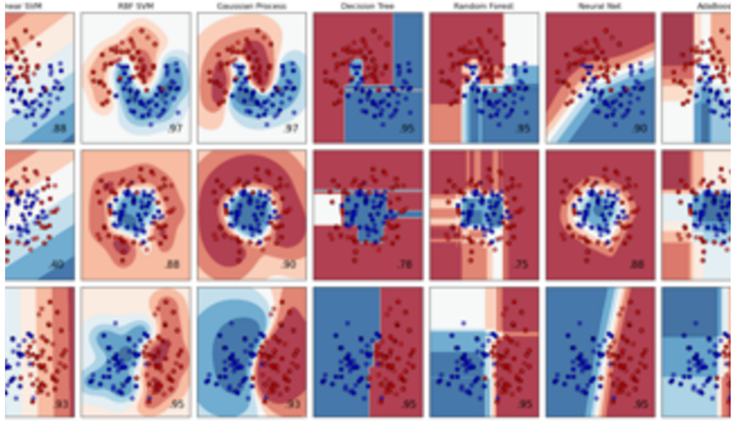
    if self._p != 1:
        prob = cp.Problem(cp.Minimize(0.5 * cp.quad_form(x, P)
                                      - cp.sum(x)
                                      + self._theta * cp.sum(cp.power(x, self._gamma))),
                          [G @ x <= h, A @ x == b])
    else:
        prob = cp.Problem(cp.Minimize(0.5 * cp.quad_form(x, P)
                                      - cp.sum(x)),
                          [G @ x <= h, A @ x == b])
    prob.solve(solver=cp.GUROBI)

    return np.ravel(x.value)
```

Classification

Identifying which category an object belongs to.

Applications: Spam detection, image recognition.
Algorithms: Gradient boosting, nearest neighbors, random forest, logistic regression, and more...

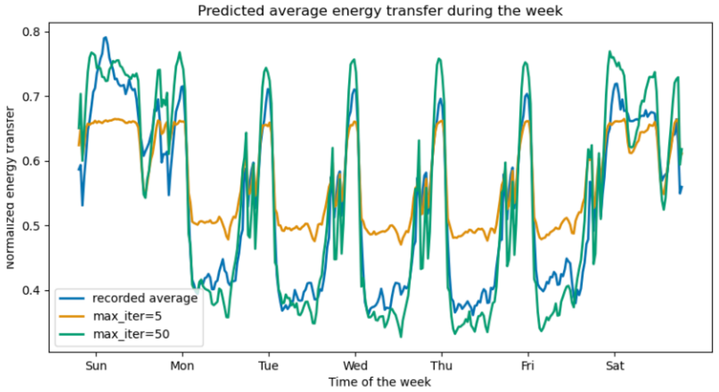


Examples

Regression

Predicting a continuous-valued attribute associated with an object.

Applications: Drug response, stock prices.
Algorithms: Gradient boosting, nearest neighbors, random forest, ridge, and more...

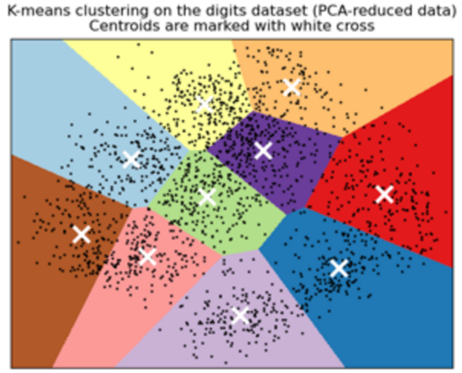


Examples

Clustering

Automatic grouping of similar objects into sets.

Applications: Customer segmentation, grouping experiment outcomes.
Algorithms: k-Means, HDBSCAN, hierarchical clustering, and more...



Examples

Dimensionality reduction

Reducing the number of random variables to consider.

Applications: Visualization, increased efficiency.
Algorithms: PCA, feature selection, non-negative matrix factorization, and more...

Model selection

Comparing, validating and choosing parameters and models.

Applications: Improved accuracy via parameter tuning.
Algorithms: Grid search, cross validation, metrics,

Preprocessing

Feature extraction and normalization.

Applications: Transforming input data such as text for use with machine learning algorithms.
Algorithms: Preprocessing, feature extraction, and more...

```
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn import svm
from sklearn.model_selection import KFold
from sklearn.ensemble import RandomForestClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.ensemble import GradientBoostingClassifier
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import AdaBoostClassifier
from sklearn.naive_bayes import GaussianNB
import matplotlib.pyplot as plt
```

```python
def test_rbf(train_sample, train_label, test_sample, test_label, C, p, skl=False, debug=False, skl_type="svm", vis=False):
    multiple_run = False

    if skl:
        if skl_type == "svm":
            clf = svm.SVC(C=C)
        elif skl_type == "knn":
            clf = KNeighborsClassifier(n_neighbors=3)
        else:
            multiple_run = True
        if not multiple_run:
            clf.fit(train_sample, train_label)
            pred = lambda x: clf.predict([x])[0]
    else:
        avg_acc = 0
        runs = 20
        for k in range(runs):
            if skl_type == "LDA":
                clf = LinearDiscriminantAnalysis()
            if skl_type == "random-forest":
                clf = RandomForestClassifier()
            if skl_type == "grad-boosting":
                clf = GradientBoostingClassifier()
            if skl_type == "decision-tree":
                clf = DecisionTreeClassifier()
            if skl_type == "adaboost":
                clf = AdaBoostClassifier(n_estimators=50, algorithm="SAMME")
            if skl_type == "gaussian-nb":
                clf = GaussianNB()
            clf.fit(train_sample, train_label)
            pred = lambda x: clf.predict([x])[0]
            avg_acc += np.mean([pred(x) == y for x, y in zip(test_sample, test_label)]) / runs
        print(f"Test accuracy: {round(avg_acc * 100, 2)} (avg, Scikit-learn {skl_type}, {runs} runs)")
        return avg_acc
```

使用 Iris 数据集做 KNN

In [334…

```
import pandas as pd
df = pd.read_csv("iris.csv")
print(df.head())
```

```
   sepal_length  sepal_width  petal_length  petal_width species
0           5.1          3.5           1.4          0.2  setosa
1           4.9          3.0           1.4          0.2  setosa
2           4.7          3.2           1.3          0.2  setosa
3           4.6          3.1           1.5          0.2  setosa
4           5.0          3.6           1.4          0.2  setosa
```

In [335…

```
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.preprocessing import MinMaxScaler
import numpy as np

# 将读入的数据分为 data 和 label
X = df.to_numpy()[:, :4]
y = df.to_numpy()[:, 4]

# 进行数据集划分
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

# 将数据全部缩放到 [0, 1] 上，依据 X_train 的最值
scaler = MinMaxScaler(feature_range=(0, 1))
X_min = np.min(X_train, axis=0)
X_max = np.max(X_train, axis=0)
X_train = scaler.fit_transform(X_train)

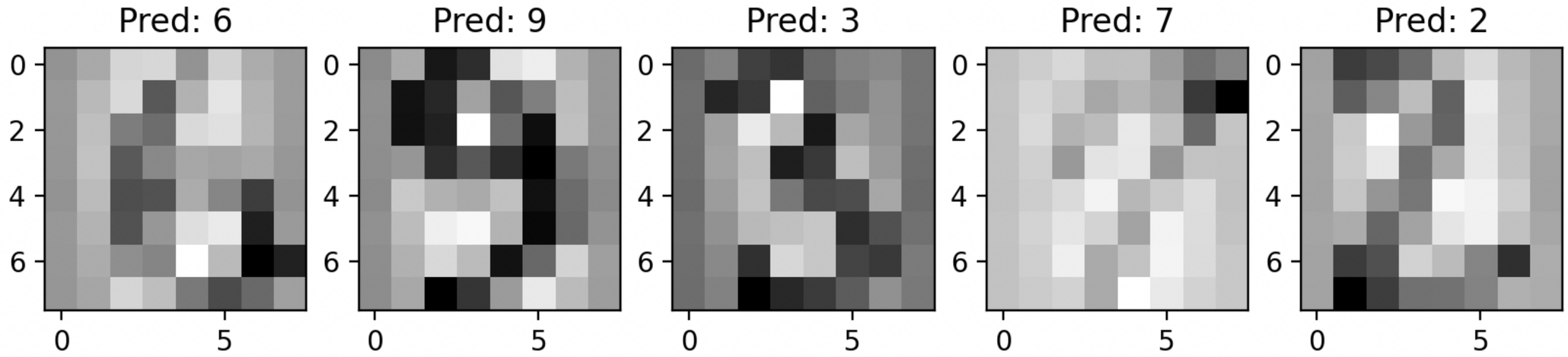
# 构建分类器
cls = KNeighborsClassifier(n_neighbors=5)

# 训练分类器
cls.fit(X_train, y_train)

# 对新的数据进行分类
def get_pred(sepal_length, sepal_width, petal_length, petal_width):
    # 将数据全部缩放到 [0, 1]
    sepal_length = (sepal_length - X_min[0]) / (X_max[0] - X_min[0])
    sepal_width = (sepal_width - X_min[1]) / (X_max[1] - X_min[1])
    petal_length = (petal_length - X_min[2]) / (X_max[2] - X_min[2])
    petal_width = (petal_width - X_min[3]) / (X_max[3] - X_min[3])
    return cls.predict([[sepal_length, sepal_width, petal_length, petal_width]])[0]

# 测试分类器在 test 集上的准确率
y_pred = []
for data in X_test:
    y_pred.append(get_pred(data[0], data[1], data[2], data[3]))
print("在测试集上的准确率：", np.mean(y_pred == y_test))
```

使用 Logistic 回归识别手写数字



使用 Gurobi 求解 0-1 背包

- 每个元素有一个价值和重量
- 背包容量有限
- 最大化价值

使用 CVXPY 求解经济学问题

- 若干个商品，价格为 pi
- 预算约束 p1x1+…+pmxm <= M
- 良好偏好（蕴含凸性）max Utility
- 例如，U(x1, …, xn) = sum ln(1+xi)

- 合适的方式：凸优化器